

Sincronizzazione tra thread

Atomicità e mutua esclusione

Roberto Marino, PhD

Università di Messina
CdL in Informatica — Reti e Sistemi Distribuiti Mod.B

6 maggio 2026



UniMe
1548

Codice apparentemente innocuo

```
1  int counter = 0;
2
3  void *increment(void *arg) {
4      for (int i = 0; i < 1000000; i++)
5          counter++;          /* <- problema */
6      return NULL;
7  }
8
9  int main(void) {
10     pthread_t t1, t2;
11     pthread_create(&t1, NULL, increment, NULL);
12     pthread_create(&t2, NULL, increment, NULL);
13     pthread_join(t1, NULL);
14     pthread_join(t2, NULL);
15     printf("counter = %d\n", counter);
16 }
```

Risultato atteso vs reale

Atteso: 2000000

Esec. 1: 1346219

Esec. 2: 1893041

Esec. 3: 1999971

Il risultato cambia ad ogni esecuzione.

Il bug non è nel codice C.

È nell'assunzione che `counter++` sia indivisibile.

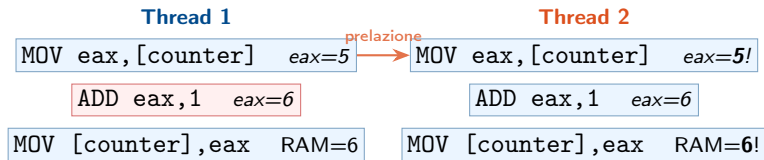
Una sola riga C diventa **tre istruzioni**:

```
; gcc -O0: counter++  
MOV eax, [counter] ; 1. LOAD  
ADD eax, 1          ; 2. ADD  
MOV [counter], eax  ; 3. STORE
```

Tra il LOAD e lo STORE il valore in RAM **non è ancora aggiornato**.

Il sistema operativo può sospendere il thread in qualsiasi punto — anche tra istruzione 1 e istruzione 2.

++counter: un interleaving disastroso!



Un incremento è perso: due thread scrivono 6!

Operazione atomica

Avviene *istantaneamente* dal punto di vista degli altri core: nessuno vede uno stato intermedio.

Il prefisso LOCK chiede alla CPU di **acquisire il bus di memoria** per tutta la durata dell'istruzione. Nessun altro core può accedere a quell'indirizzo nel frattempo.

; Non atomico: 3 istruzioni separate

```
MOV  eax, [counter]
```

```
ADD  eax, 1
```

```
MOV  [counter], eax
```

; Atomico: una sola operazione indivisibile

```
LOCK ADD [counter], 1
```

Istruzioni che accettano LOCK

LOCK ADD	somma atomica
LOCK XCHG	swap atomico
LOCK XADD	fetch-and-add
LOCK CMPXCHG	compare-and-swap
LOCK INC/DEC	inc/dec atomico

Nota

LOCK ADD risolve il contatore, ma non restituisce il valore precedente. Per costruire un lock servono XCHG e CMPXCHG.

Cosa fa XCHG reg, [mem]

Scambia il contenuto del **registro** con quello della **locazione di memoria**, atomicamente.

Prima: reg = A, [mem] = B

Dopo: reg = B, [mem] = A

```
; Esempio: scambia eax con [lock_var]
; Supponiamo eax=1, [lock_var]=0
```

```
XCHG eax, [lock_var]
```

```
; Dopo: eax=0, [lock_var]=1
; -- atomicamente --
```

XCHG con memoria è **sempre atomico** senza il prefisso LOCK — il processore lo garantisce per compatibilità storica.

Perché è utile?

Possiamo *scrivere un valore e leggere quello precedente* in un solo colpo atomico. Nessun altro thread può intromettersi tra la lettura e la scrittura.

Pseudocodice equivalente

```
1  /* XCHG eax, [mem] e' equivalente a: */
2  int xchg(int *mem, int val) {
3      int old = *mem; /* leggi */
4      *mem = val;     /* scrivi */
5      return old;     /* restituisci vecchio */
6      /* tutto atomicamente */
7  }
```

Questa primitiva è la base del **Test-and-Set**.

Cosa fa LOCK XADD [mem], reg

1. Scambia [mem] e reg (come XCHG)
2. Somma i due valori e scrive in [mem]

Il risultato netto: [mem] += reg, e reg contiene il valore **precedente** di [mem].

```
; Supponiamo: eax=1, [counter]=5
LOCK XADD [counter], eax

; Dopo: [counter]=6, eax=5
;      ^-- aggiornato      ^-- vecchio valore
```

Pseudocodice equivalente (C)

```
1  /* LOCK XADD [mem], n
2     e' equivalente a: */
3  int fetch_and_add(int *mem, int n) {
4      int old = *mem; /* leggi */
5      *mem = old + n; /* aggiorna */
6      return old;    /* vecchio valore */
7      /* tutto atomicamente */
8  }
```

Differenza da LOCK ADD:

LOCK ADD incrementa ma *non ritorna* il valore precedente. LOCK XADD sì — e questo fa tutta la differenza per costruire strutture come il **ticket lock**.

Cosa fa LOCK CMPXCHG [mem], reg

1. Confronta [mem] con eax (sempre il registro eax)
2. **Se uguali:** scrive reg in [mem], imposta il flag ZF=1
3. **Se diversi:** copia [mem] in eax, imposta il flag ZF=0

In entrambi i casi: atomico.

```
; Vogliamo: se [lock]==0, scrivi 1
MOV  eax, 0          ; expected = 0
MOV  ecx, 1          ; desired  = 1
LOCK CMPXCHG [lock], ecx

; Se [lock] era 0: [lock]=1, ZF=1 (successo)
; Se [lock] era 1: eax=1, ZF=0 (fallimento)
JNZ  retry          ; riprova se ZF=0
```

Pseudocodice equivalente (C)

```
1  /* LOCK CMPXCHG [mem], desired
2     (con eax = expected)
3     e' equivalente a: */
4  bool cas(int *mem,
5           int expected,
6           int desired) {
7      if (*mem == expected) {
8          *mem = desired;
9          return true;    /* ZF=1 */
10     }
11     /* *mem invariato */
12     return false;       /* ZF=0 */
13     /* tutto atomicamente */
14 }
```

In C moderno:

atomic_compare_exchange_weak() da
<stdatomic.h> compila esattamente in
questa istruzione su x86.

Test-and-Set (TAS): semantica

TAS è una **primitiva di sincronizzazione** costruita sopra XCHG. Non è un'istruzione assembler: è un'operazione concettuale.

Cosa fa TAS(ptr)

1. Legge il valore corrente di *ptr
2. Scrive **sempre 1** in *ptr
3. Restituisce il valore *precedente*

Tutto in modo **atomico**.

```
; Implementazione x86 con XCHG
test_and_set:
    MOV  al, 1          ; scrivi sempre 1
    XCHG al, [ptr]      ; swap atomico
    ; al = valore precedente di [ptr]
    RET
```

Pseudocodice C equivalente

```
1  /* TAS atomica -- pseudocodice */
2  int test_and_set(int *ptr) {
3      int old = *ptr; /* leggi */
4      *ptr = 1;      /* scrivi 1 */
5      return old;    /* vecchio valore */
6      /* tutto atomicamente */
7  }
```

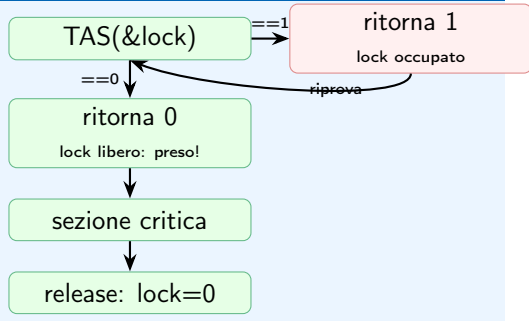
Come interpretare il valore restituito:

- 0 il lock era **libero**: lo abbiamo preso noi
- 1 il lock era **occupato**: qualcun altro ce l'ha

TAS spinlock: il primo lock possibile

```
1  /* In C moderno: __sync_lock_test_and_set
2     e' il TAS fornito da GCC.
3     Compila in XCHG su x86. */
4
5  int lock = 0; /* 0 = libero, 1 = occupato */
6
7  void acquire(int *lock) {
8      /* Aspetta finche' TAS restituisce 0
9       * (il lock era libero e lo abbiamo preso)
10         */
11     while (__sync_lock_test_and_set(lock, 1) ==
12            1)
13         ; /* busy-wait: gira nel loop */
14 }
15
16 void release(int *lock) {
17     /* Scrivi 0: libera il lock.
18      * __sync_lock_release garantisce
19      * il memory barrier necessario. */
20     __sync_lock_release(lock);
21 }
22
23 /* Uso */
24 acquire(&lock);
25 counter++; /* sezione critica */
26 release(&lock);
```

Come funziona



Il difetto strutturale di TAS

TAS **scrive sempre**, anche quando il lock è occupato: ogni iterazione del busy-wait esegue XCHG, che blocca il bus.

Con N thread: N scritture per ciclo. La cache line rimbalza tra i core.

Problema più profondo: starvation

Con TAS, quando il lock si libera, **tutti** tentano XCHG insieme. Chi vince è casuale: un thread può aspettare indefinitamente (**starvation**).

Miglioramento: test-then-TAS

```
1 void acquire(int *lock) {
2     while (1) {
3         while (*lock == 1) ; /* solo lettura */
4         if (__sync_lock_test_and_set(lock,1)
5             ==0)
6             return; /* preso! */
7     }
```

La soluzione: equità

Serve una primitiva che assegni un *turno numerato* a ogni thread. Questa è la funzione di **Fetch-and-Add**.

Cosa fa Fetch-and-Add(*ptr*, *n*)

1. Legge il valore corrente di **ptr*
2. Aggiunge *n* a **ptr*
3. Restituisce il valore *precedente*

Tutto in modo **atomico**.

```
; Implementazione x86: LOCK XADD
; Supponiamo: eax=1, [next]=7
LOCK XADD [next], eax
; Dopo: [next]=8, eax=7 (vecchio valore)
```

```
1 /* In C con <stdatomic.h> */
2 atomic_int x = 0;
3 int old = atomic_fetch_add(&x, 1);
4 /* old = valore prima dell'incremento */
```

Differenza chiave rispetto a TAS:

TAS	scrive <i>sempre 1</i> , restituisce il vecchio valore
FAA	<i>aggiunge n</i> , restituisce il vecchio valore

L'intuizione

Se tre thread chiamano

`fetch_and_add(&x, 1)` con `x=0`

contemporaneamente, ottengono **0, 1, 2**
— valori *distinti*, ognuno unico, ordinati.

È come distribuire biglietti numerati: ogni chiamata prende il proprio numero senza sovrapposizioni.

Ticket spinlock: equità garantita con FAA

```
1  #include <stdatomic.h>
2
3  typedef struct {
4      atomic_int owner; /* a chi tocca ora */
5      atomic_int next; /* prossimo biglietto */
6  } ticket_lock_t;
7
8  void ticket_acquire(ticket_lock_t *l) {
9      /* Prendo il mio biglietto (FAA atomico) */
10     int my_turn = atomic_fetch_add(&l->next, 1);
11
12     /* Aspetto che sia il mio turno */
13     while (atomic_load(&l->owner) != my_turn)
14         ; /* busy-wait */
15 }
16
17 void ticket_release(ticket_lock_t *l) {
18     /* Avanza il turno: il prossimo in coda
19      * vede il suo numero e parte */
20     atomic_fetch_add(&l->owner, 1);
21 }
22
23 /* Uso */
24 ticket_lock_t lk = { .owner=0, .next=0 };
25 ticket_acquire(&lk);
26 counter++;
27 ticket_release(&lk);
```

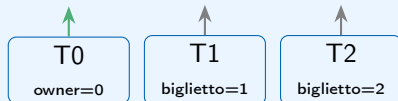
Equità garantita

owner=0, next=3

va avanti

aspetta

aspetta



Ogni thread ha un biglietto diverso.

owner avanza uno alla volta. Nessuna starvation possibile.

Limite residuo

Tutti i thread leggono owner in busy-wait — la cache line rimbalza ancora tra i core. Risolto dai **MCS/qspinlock** usati nel kernel.

Cosa fa *CAS(ptr, expected, desired)*

1. Confronta **ptr* con *expected*
2. **Solo se uguali:** scrive *desired* in **ptr* e ritorna *true*
3. **Se diversi:** non tocca nulla e ritorna *false*

Tutto in modo **atomico**.

```
; x86: LOCK CMPXCHG [mem], ecx
; (eax = expected, ecx = desired)
MOV  eax, 0          ; expected = 0
MOV  ecx, 1          ; desired  = 1
LOCK CMPXCHG [lock], ecx
; ZF=1: [lock] era 0, ora e' 1 (successo)
; ZF=0: [lock] non era 0, eax=valore attuale
```

Pseudocodice C equivalente

```
1  bool cas(int *ptr,
2         int  expected,
3         int  desired) {
4      if (*ptr == expected) {
5          *ptr = desired;
6          return true;
7      }
8      return false;
9      /* tutto atomicamente */
10 }
11
12 /* In C con <stdatomic.h> */
13 int old = 0;
14 bool ok = atomic_compare_exchange_weak(
15         &lock, &old, 1);
16 /* ok==true: lock acquisito */
17 /* ok==false: old aggiornato al valore reale
   */
```

CAS vs TAS: perché CAS è più potente

TAS scrive *sempre*:

```
1  /* TAS: scrivo 1 a prescindere */
2  int old = test_and_set(&lock);
3  /* se old==1 il lock era occupato
4     MA ho comunque scritto 1 sul bus */
```

CAS scrive *solo se necessario*:

```
1  /* CAS: scrivo solo se trovo 0 */
2  int expected = 0;
3  bool ok = atomic_compare_exchange_weak(
4      &lock, &expected, 1);
5  /* se ok==false: NON ho scritto nulla
6     -> nessun traffico sul bus
7     -> cache line non invalidata */
```

Spinlock con CAS: meno traffico

```
1  void acquire(atomic_int *lock) {
2      int expected;
3      do {
4          /* Leggo finche' trovo 0 */
5          while (atomic_load(lock) != 0)
6              ; /* solo letture: economico */
7          expected = 0;
8          /* Tenta CAS: scrivo solo se 0 */
9      } while (!atomic_compare_exchange_weak(
10         lock, &expected, 1));
11 }
```

Il vero vantaggio di CAS

CAS permette operazioni **lock-free** impossibili con TAS: aggiornare una struttura dati solo se non è cambiata dall'ultima lettura (stack, queue, counter lock-free).

Il limite comune di tutti gli spinlock

TAS spinlock, ticket spinlock, CAS spinlock: tutti condividono lo stesso problema di fondo.

Busy-waiting

Un thread che non riesce ad acquisire il lock **non si addormenta**: gira in un loop consumando il 100% del suo timeslice CPU.

- ▶ Spreca cicli di CPU
- ▶ Rallenta gli altri thread (stesso core)
- ▶ Peggiora con sezioni critiche lunghe

La soluzione: dormire nel kernel

Se il lock è occupato, il thread dovrebbe **dormire** — cedere la CPU — e venire *risvegliato* solo quando il lock si libera.

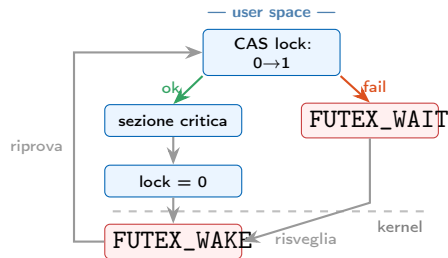
Questo richiede il **kernel**, l'unico che può sospendere e risvegliare i thread.

Ma ogni syscall costa migliaia di cicli: non conveniente se il lock è spesso libero.

La soluzione ideale

Zero syscall quando il lock è libero. Kernel solo quando è necessario dormire. Questo è il **futex**.

Futex: Fast Userspace muTEX



L'idea in tre passi

1. **Fast path** — lock libero: CAS in user space, zero syscall
2. **Slow path** — lock occupato: FUTEX_WAIT sospende il thread
3. **Unlock**: scrive 0, chiama FUTEX_WAKE

Mutex POSIX: pthread_mutex_t

```
1 #include <pthread.h>
2
3 /* Inizializzazione statica */
4 pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
5 /* oppure dinamica */
6 pthread_mutex_init(&mtx, NULL);
7
8 /* Acquisisci (blocca se occupato) */
9 pthread_mutex_lock(&mtx);
10
11 /* Prova senza bloccare */
12 int rc = pthread_mutex_trylock(&mtx);
13 /* rc==0: acquisito rc==EBUSY: occupato */
14
15 /* Rilascia */
16 pthread_mutex_unlock(&mtx);
17
18 /* Distruggi (solo dopo unlock) */
19 pthread_mutex_destroy(&mtx);
20 /* gcc -Wall -pthread -o prog prog.c */
```

Cosa c'è sotto: futex

pthread_mutex_lock usa il pattern futex: CAS in user space, FUTEX_WAIT solo se necessario. Zero overhead se libero.

Tipi (pthread_mutexattr_t)

NORMAL	default, no check
ERRORCHECK	errore se ri-lockato
RECURSIVE	stesso thread può ri-lockare n volte

Portabilità

Standard POSIX: Linux, macOS, BSD. Usa il meccanismo ottimale per piattaforma.

Esempio completo: contatore con mutex

```
1  #include <pthread.h>
2  #include <stdio.h>
3
4  #define N      4          /* numero di thread */
5  #define ITE 1000000      /* iterazioni per thread */
6
7  int counter = 0;
8  pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
9
10 void *worker(void *arg) {
11     for (int i = 0; i < ITE; i++) {
12         pthread_mutex_lock(&mtx);    /* entra in sezione critica */
13         counter++;
14         pthread_mutex_unlock(&mtx);  /* esce dalla sezione critica */
15     }
16     return NULL;
17 }
18
19 int main(void) {
20     pthread_t t[N];
21     for (int i = 0; i < N; i++) pthread_create(&t[i], NULL, worker, NULL);
22     for (int i = 0; i < N; i++) pthread_join(t[i], NULL);
23     printf("counter = %d (atteso %d)\n", counter, N * ITE);
24     pthread_mutex_destroy(&mtx);
25 }
```

`gcc -Wall -pthread -o counter counter.c` $\Rightarrow$ counter = 4000000 (atteso
4000000) sempre.

1. Dimenticare unlock

```
1 pthread_mutex_lock(&mtx);
2 if (errore) return -1; /* lock mai rilasciato!
   */
3 fai_lavoro();
4 pthread_mutex_unlock(&mtx);
```

Ogni uscita dalla funzione deve passare per unlock. In C usare goto cleanup per gestire tutti i rami.

2. Deadlock da lock annidati

```
1 /* Thread 1 */      /* Thread 2 */
2 lock(A);             lock(B);
3 lock(B);             lock(A); /* deadlock! */
```

Regola: tutti i thread devono acquisire i lock nello stesso ordine.

3. Variabile condivisa senza lock

```
1 /* Thread 1 */      /* Thread 2 */
2 lock(&mtx);
3 x = 42;              if (x==42) /* ERRORE! */
4 unlock(&mtx);         fai();
```

Ogni accesso, anche in sola lettura, deve essere protetto da lock. Usare pthread_rwlock_t per molti lettori.

4. Distruggere un mutex locked

```
1 pthread_mutex_lock(&mtx);
2 pthread_mutex_destroy(&mtx); /* UB! */
```

Comportamento indefinito. Distruggere solo dopo unlock, quando nessun thread aspetta.

Inizializzazione del mutex: statica e dinamica

Inizializzazione statica (variabile globale)

```
1  /* Funziona solo per variabili globali
2   * o statiche -- non per quelle allocate
3   * dinamicamente con malloc(). */
4  pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER
5      ;
6  /* Pronto all'uso: nessuna chiamata
7   * a pthread_mutex_init() necessaria. */
8  pthread_mutex_lock(&mtx);
9  counter++;
10 pthread_mutex_unlock(&mtx);
11
12 /* Distruggi sempre prima di uscire */
13 pthread_mutex_destroy(&mtx);
```

Inizializzazione dinamica (heap o stack)

```
1  /* Necessaria per mutex allocati con
2   * malloc() o dichiarati in struct. */
3  pthread_mutex_t *mtx = malloc(
4      sizeof(pthread_mutex_t));
5
6  pthread_mutex_init(mtx, NULL);
7  /* NULL = attributi di default:
8   * tipo NORMAL, non ricorsivo */
9
10 pthread_mutex_lock(mtx);
11 counter++;
12 pthread_mutex_unlock(mtx);
13
14 pthread_mutex_destroy(mtx);
15 free(mtx);
```

Destroy obbligatorio

pthread_mutex_destroy() libera le risorse

pthread_mutex_timedlock: acquisizione con timeout

```
1 #include <pthread.h>
2 #include <time.h>
3 #include <errno.h>
4
5 pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
6
7 void tenta_acquisizione(void) {
8     /* 1. Leggi l'ora corrente (tempo assoluto) */
9     struct timespec ts;
10    clock_gettime(CLOCK_REALTIME, &ts);
11
12    /* 2. Aggiungi il timeout desiderato */
13    ts.tv_sec += 3; /* aspetta al massimo 3 s */
14
15    /* 3. Tenta l'acquisizione con scadenza */
16    int rc = pthread_mutex_timedlock(&mtx, &ts);
17
18    if (rc == 0) {
19        /* lock acquisito: sezione critica */
20        fai_lavoro();
21        pthread_mutex_unlock(&mtx);
22    } else if (rc == ETIMEDOUT) {
23        /* scaduto: NON chiamare unlock */
24        fprintf(stderr, "timeout: rinuncio\n");
25    }
26 }
```

Tempo assoluto, non relativo

timedlock vuole un istante preciso nel tempo, non una durata. Bisogna leggere l'ora corrente con `clock_gettime(CLOCK_REALTIME)` e sommare i secondi di attesa desiderati.

Valori di ritorno

0	lock acquisito
ETIMEDOUT	scaduto il timeout
EDEADLK	deadlock rilevato
EINVAL	parametri non validi

Non usare CLOCK_MONOTONIC

timedlock accetta solo

pthread_rwlock_t: lettori e scrittori

```
1 #include <pthread.h>
2
3 pthread_rwlock_t lk = PTHREAD_RWLOCK_INITIALIZER;
4 /* oppure: pthread_rwlock_init(&lk, NULL) */
5
6 /*          Lettore: accesso condiviso          */
7 void leggi(void) {
8     pthread_rwlock_rdlock(&lk);
9     /* piu' lettori possono stare qui insieme */
10    printf("%s\n", dato_condiviso);
11    pthread_rwlock_unlock(&lk);
12 }
13
14 /*          Scrittore: accesso esclusivo          */
15 void scrivi(const char *nuovo) {
16     pthread_rwlock_wrlock(&lk);
17     /* nessun altro lettore o scrittore attivo */
18     strcpy(dato_condiviso, nuovo);
19     pthread_rwlock_unlock(&lk);
20 }
21
22 /*          Pulizia          */
23 /* pthread_rwlock_destroy(&lk); */
```

Regole di accesso

	rdlock	wrlock
Nessuno attivo	✓	✓
Lettori attivi	✓	blocca
Scrittore attivo	blocca	blocca

Quando usarlo

Conveniente quando le **letture sono molto più frequenti** delle scritture: rubrica, cache, tabelle di configurazione. Se letture e scritture sono bilanciate, un mutex ordinario ha meno overhead.

unlock è unico